

Vorlage 5.21-E — Handprüfungen automatisieren

Vorlage 5.21-E — Handprüfungen automatisieren

Auftraggeber: Adam, 27.07.2026, 09:59 Uhr (Sprachnachricht)
Ausgearbeitet von: Claudia (VPS-Bot-Sitzung, nur Leserecht am Repo) **Umzusetzen von:** Mick (Migrationssitzung am Mac) **Stand der Vorlage:** 27.07.2026, 11:40 Uhr

Änderungshistorie

2026-07-27 11:40 — Abschnitt 11 ergänzt. Entstand aus Adams Frage von 11:29 Uhr („wie denkst du selbst um diese Ecken?“): Seine Methode — rückwärts vom Ziel statt vorwärts vom Auftrag — **einmal angewandt und dabei zwei weitere stille Stellen gefunden:** kein Wächter über die Zeitgeber, und ein verschluckter Versandfehler. **2026-07-27 11:25** — Abschnitt 10 ergänzt (wegbrechende Quellen) nach Adams Nachricht von 11:21 Uhr. Zweiter **verifizierter Befund:** Eine nicht erreichbare Quelle wird heute nur ins Protokoll geschrieben, **nie gemeldet** — die Überwachung kann still einschlafen. **2026-07-27 11:20** — Abschnitt 9 ergänzt (Geltung für alles Künftige) nach Adams Nachricht von 11:11 Uhr. Enthält einen **verifizierten Befund:** Der Vollständigkeits-Wächter R2 deckt eigene Skripte ab, das **Komponenten-Register hat keinen Prüfer**, und die Modul-Liste des Wächters ist fest verdrahtet. **2026-07-27 10:40** — Abschnitt 8 ergänzt (Ausbaustufen und Regelwerk) nach Adams Sprachnachricht von 10:23 Uhr: Die Vollautomatik der Verfahrensprüfung wird **nicht verworfen**, sondern als Ausbaustufe 2 geführt; sein zweiter, eigener Grund für den

Freigabeknopf (bewusstes Mitverfolgen) in 4.3 nachgetragen; das Regelwerk für selbsttätige Aktualisierung als Ausbaustufe 3 aufgenommen. **2026-07-27 10:35** — Erstfassung.

1. Auftrag in Adams Worten

Nach der Monitor-Meldung vom Montag, 27.07., 04:20 Uhr fiel Adam auf, dass neun Einträge im Komponenten-Register als `manual` geführt werden. Seine Vorgabe:

„Manuell prüfen heißt in dem Fall aber nicht, dass ich das machen muss, sondern dass du das manuell durchprüfst — was im Endeffekt für mich dann aufs Gleiche hinauskommt. Das musst du dann in den Updater einbinden bitte, dass die ganzen Sachen geprüft werden, zumindest in regelmäßigen Abständen, sagen wir mal wöchentlich. ... Wichtig ist nur, dass ich das nicht machen muss. Von außen darf das genauso aussehen wie der Updater. Und diesen Monitorlauf mit prüfen, ob es inzwischen ein besseres Verfahren gibt — das reicht einmal im Monat. Wir müssen jetzt nicht sofort drauf springen, sobald was Neues kommt.“

Kern: Was ohne Urteilskraft prüfbar ist, prüft der Monitor mit — wöchentlich, im bestehenden Takt (Mo 04:20). Die zwei langsamen Punkte laufen monatlich. Für Adam ändert sich nichts an der Bedienung: Er bekommt weiterhin nur dann eine Nachricht, wenn es etwas gibt.

2. Ist-Stand (im Code geprüft, 27.07.2026)

Stück	Stand
<code>scripts/version_monitor.py</code>	drei Prüffarten gebaut: <code>pip</code> , <code>npm_global</code> , <code>node</code>
<code>components.json</code>	20 Einträge, davon 9 × <code>manual</code> (nur Erinnerungstext)
<code>claude-version-monitor.timer</code>	<code>OnCalendar=Mon *-*-* 04:20:00</code> , <code>Persistent=true</code> — Takt stimmt bereits
Manual-Erinnerung	

Stück	Stand
	wird nur angehängt, wenn ohnehin eine Meldung rausgeht (Z. 179 f.)

Befund A — dokumentierte, aber nie gebaute Prüfarm. Der Registerkopf nennt unter `_kinds` eine Art `github_release` (`repo`, `current_cmd`). In `HANDLERS` (Z. 104–108) existiert sie **nicht**. Ein Eintrag mit dieser Art landet in Zeile 149 („unbekannter kind“) und wird **still übersprungen** — er stünde im Register und würde nie geprüft. Klassischer Fall „dokumentiert ≠ gebaut“; wird mit diesem Auftrag geschlossen.

Befund B — Rückwirkung auf den Updater (wichtig!).

`updater.py` Z. 63 lautet:

```
if kind == "manual" or kind not in vm.HANDLERS:
    continue
```

Der Updater filtert also **negativ** gegen `HANDLERS`. Sobald dort `apt`, `github_release` oder `docker_image` auftauchen, erscheinen diese Komponenten in der Freigabeliste des Updaters, obwohl er sie nicht einspielen kann (Z. 146–152 antwortet dann „kind nicht installierbar“). **Ohne die Korrektur in Teil 4 wird die Freigabeliste nach dem Umbau unbrauchbar.**

3. Einteilung der neun Handpunkte

Komponente	neue Art	Takt	Quelle
pandoc	<code>apt</code>	wöchentlich	lokale Paketverwaltung
weasyprint	<code>apt</code>	wöchentlich	lokale Paketverwaltung
docker.io	<code>apt</code>	wöchentlich	lokale Paketverwaltung
ffmpeg	<code>apt</code>	wöchentlich	lokale Paketverwaltung
debian	<code>apt</code> (Stellvertreter <code>base-files</code>)	wöchentlich	lokale Paketverwaltung
whisper.cpp	<code>github_release</code>	wöchentlich	GitHub-Releases + lokaler git-Stand
lobe-chat	<code>docker_image</code>	wöchentlich	


```
except Exception:
    return ""
```

(b) Prüffart `apt` — rein lokal, kein Netz, sprachneutral:

```
_APT_CACHE: dict[str, tuple[str, str]] = {}

def _apt_policy(paket: str) -> tuple[str, str]:
    """(installiert, kandidat). LC_ALL=C erzwingt englische
        Feldnamen."""
    if paket in _APT_CACHE:
        return _APT_CACHE[paket]
    out = _run(["apt-cache", "policy", paket], {"LC_ALL": "C"})

    def _feld(name: str) -> str:
        m = re.search(rf"^\s*{name}:\s*(\S+)", out, re.MULTILINE)
        v = m.group(1).strip() if m else ""
        return "" if v in ("(none)", "(keine)") else v

    paar = (_feld("Installed"), _feld("Candidate"))
    _APT_CACHE[paket] = paar
    return paar

def cur_apt(comp: dict) -> str:
    return _apt_policy(comp["ref"])[0]

def latest_apt(comp: dict) -> str:
    return _apt_policy(comp["ref"])[1]
```

(c) Prüffart `github_release` — Marke, sonst Datum:

```
def latest_github(comp: dict) -> str:
    d = _get_json(f"https://api.github.com/repos/{comp['repo']}/
        releases/latest")
    return (d or {}).get("tag_name", "") if d else ""

def cur_github(comp: dict) -> str:
    """Installierter Stand eines git-Klons. Befehl wird FEST
        gebaut – nie aus
        dem Register (kein Freitext-Befehl, siehe Sicherheitsteil)."""
```

```

pfad = comp.get("git_path", "")
if not pfad or not Path(pfad, ".git").is_dir():
    return ""
marke = _run(["git", "-C", pfad, "describe", "--tags", "--abbrev=0"]).strip()
return marke # leer bei flachem Klon → Datumsweg unten greift

```

Für den flachen Klon zusätzlich ein Datumsvergleich, der in `main()` greift, wenn `cur` leer bleibt:

```

def _github_neuer_als_klon(comp: dict) -> tuple[bool, str, str]:
    """(neuer, lokales_datum, release_datum) – für Klone ohne Marken."""
    pfad = comp.get("git_path", "")
    lokal = _run(["git", "-C", pfad, "log", "-1", "--format=%CI"]).strip()[:10]
    d = _get_json(f"https://api.github.com/repos/{comp['repo']}/releases/latest")
    rel = ((d or {}).get("published_at") or "")[:10]
    if not lokal or not rel:
        return (False, lokal, rel)
    return (rel > lokal, lokal, rel)

```

(d) Prüftart `docker_image`:

```

def cur_docker(comp: dict) -> str:
    out = _run(["docker", "image", "inspect", comp["ref"], "--format",
        '{{index .Config.Labels "org.opencontainers.image.version"}}'])
    v = out.strip()
    return "" if v in ("", "<no value>") else v

```

```

def latest_docker(comp: dict) -> str:
    return latest_github(comp) # Register trägt zusätzlich `repo`

```

Das führende „v“ der GitHub-Marken stört nicht: `_vtuple` zieht nur Ziffern.

(e) Prüftart `modelle` — Mengenabgleich, **kein Modelllauf**:

```

def _verfuegbare_modelle() -> list[str]:
    """Öffentliche Modell-Liste. Weg 1: Auflistungs-Endpunkt (kostet keine

```

*Token, braucht einen Schlüssel). Weg 2: öffentliche
Übersichtsseite.*

*Keine Quelle erreichbar → leere Liste → Meldung 'Quelle n/
a'.'''*

```
key = _env_wert("ANTHROPIC_API_KEY")
if key:
    req = urllib.request.Request(
        "https://api.anthropic.com/v1/models",
        headers={"x-api-key": key, "anthropic-version":
            "2023-06-01"})
    try:
        with urllib.request.urlopen(req, timeout=HTTP_TIMEOUT)
        as r:
            d = json.loads(r.read().decode("utf-8"))
            return [m.get("id", "") for m in d.get("data", [])]
    except Exception:
        pass
roh = _http_text("https://docs.claude.com/en/docs/about-
claude/models/overview")
return sorted(set(re.findall(r"claude-[a-z0-9][a-z0-9.\-]
{2,40}", roh)))
```

```
def _hinterlegte_aliasse() -> dict[str, str]:
    """Liest _MODEL_ALIASES aus bot.py per Muster – KEIN Import
    (der Bot darf
    dabei nicht anlaufen). Greift das Muster nicht, meldet der
    Monitor
    'Quelle n/a' statt zu raten."""
    txt = (ROOT / "bot.py").read_text(encoding="utf-8",
        errors="ignore")
    block = re.search(r"_MODEL_ALIASES\s*=\s*\{(.*)\}", txt,
        re.S)
    if not block:
        return {}
    return dict(re.findall(r'"([a-z]+)"\s*:\s*"([^\"]+)"',
        block.group(1)))
```

Meldung nur, wenn eine Modell-Kennung auftaucht, die keinem
hinterlegten Alias entspricht und deren Reihennummer höher ist.
Format der Meldung:

Modelle: `claude-opus-6` verfügbar — hinterlegt ist `claude-
opus-5`. Alias in `bot.py` anheben (bewusste Entscheidung,
kein Automatismus).

(f) Monatstakt. Kleine Zustandsdatei neben dem Protokoll:

```
STATE = Path(os.environ.get("VERSION_MONITOR_STATE")
              or "/home/claudebot/claude-telegram-bot/logs/version-
              monitor-state.json")
```

```
def _faellig(name: str, tage: int) -> bool:
    try:
        st = json.loads(STATE.read_text())
    except Exception:
        st = {}
    letzte = st.get(name)
    if not letzte:
        return True
    return (datetime.now() - datetime.fromisoformat(letzte)).days
    >= tage
```

```
def _quittieren(name: str) -> None:
    try:
        st = json.loads(STATE.read_text())
    except Exception:
        st = {}
    st[name] = datetime.now().isoformat(timespec="seconds")
    STATE.write_text(json.dumps(st, indent=2))
```

Komponenten mit `"intervall": "monatlich"` werden nur geprüft, wenn `_faellig(name, 28)`. Ein Weg, ein Protokoll, keine zweite Systemd-Einheit.

(g) Vergleich je Art. Debian-Versionen tragen Epochen und Bausuffixe (`7:7.1.5-0+deb13u1`); der bestehende `_cmp` über Ziffernfolgen ist dafür untauglich. Für `apt` gilt: **ungleich = Update**, weil apt den Kandidaten selbst bestimmt.

```
UNGLEICH_VERGLEICH = {"apt"}
```

```
def _major_apt(cur: str, latest: str) -> bool:
    def kopf(v: str) -> str:
        v = v.split(":", 1)[-1]
        abschneiden
        m = re.match(r"(\d+)", v)
        # Epoche
```



```

        return m.group(1) if m else ""
    return kopf(cur) != kopf(latest)

```

In `main()` an der Vergleichsstelle (heute Z. 156):

```

if kind in UNGLEICH_VERGLEICH:
    newer = bool(latest) and cur != latest
    major = newer and _major_apr(cur, latest)
else:
    newer, major = _cmp(cur, latest)

```

(h) Handpunkt-Erinnerung. Nach dem Umbau bleibt nur noch `verfahren-medien` als `manual`. Die Erinnerung wird zur **Fälligkeits-Meldung mit Knopf** (4.3) und darf dann auch ohne andere Updates rausgehen — sonst verschluckt sie sich in ruhigen Wochen (heutiges Verhalten, Z. 179 f.).

4.2 `components.json`

- `_kinds` fortschreiben: `apt (ref) | github_release (repo, git_path) | docker_image (ref, repo) | modelle | manual`. **current_cmd streichen** — es wird bewusst kein Freitext-Befehl akzeptiert.
- Die fünf apt-Einträge: `"kind": "apt" + "ref" (pandoc, weasyprint, docker.io, ffmpeg; für Debian base-files als Point-Release-Stellvertreter)`.
- `whisper.cpp`: `"kind": "github_release", "repo": "ggml-org/whisper.cpp", "git_path": "/home/claudebot/whisper.cpp"`.
- `lobe-chat`: `"kind": "docker_image", "ref": "lobehub/lobe-chat:latest", "repo": "lobehub/lobe-chat"`.
- `claude-modelle`: `"kind": "modelle", "intervall": "monatlich"`. **Notiz korrigieren** — der Satz „lässt sich NICHT offline ermitteln, deshalb bewusst manual“ stimmt nach dem Umbau nicht mehr und würde als falsche Begründung stehenbleiben.
- `verfahren-medien`: bleibt `manual`, bekommt `"intervall": "monatlich"`.

4.3 `bot.py` — Knopf für die Verfahrensprüfung

Der Monitor schickt bei Fälligkeit eine Nachricht mit einem Knopf (`reply_markup` als JSON im `sendMessage`-Aufruf — der Monitor spricht die Telegram-Schnittstelle ohnehin direkt an):

Verfahrensprüfung fällig (Bild- und Videoauswertung,
zuletzt vor 4 Wochen) [Jetzt prüfen]

Neuer Rückruf analog zum Updater — Muster `^vfp:` neben dem bestehenden `^upd:` (heute Z. 7804). Beim Druck läuft die Recherche als **Adams Auftrag** in der Bot-Sitzung; danach `_quittieren("verfahren-medien")`.

Warum der Knopf und nicht vollautomatisch — zwei voneinander unabhängige Gründe:

1. **Leitplanke.** Ein zeitgesteuerter Modelllauf über das Abo ist genau das, was `2026-07-24_agb-faktensammlung.md` Abschnitt D untersagt („zeitgetriggert Abo-Aufruf ohne Adams Zutun → Grauzone“).
2. **Adams eigener Grund (27.07., 10:23 Uhr).** Er will die Schritte, die später automatisch laufen sollen, zunächst bewusst mitbekommen — „damit ich wirklich prüfen und abgleichen kann, ob das so sinnvoll ist, ob es gut funktioniert“. Der Knopf ist damit kein Zugeständnis an eine Regel, sondern gewollte Sichtbarkeit während der Einführungszeit.

Der Knopf sieht für Adam aus wie das, was er vom Updater kennt: Meldung kommt von selbst, ein Fingertipp gibt frei. Die Vollautomatik bleibt als Ausbaustufe erhalten, siehe Abschnitt 8 — **sie wird nicht verworfen, nur zurückgestellt.**

4.4 `scripts/updater.py` — Pflichtkorrektur (Befund B)

```
INSTALLIERBAR = {"pip", "npm_global"}      # was der Updater
        wirklich einspielt
...
if kind not in INSTALLIERBAR:
    continue
```

Ersetzt die heutige Negativprüfung gegen `vm.HANDLERS` (Z. 63). Ohne diese Änderung wandern Systempakete und Fremddabbilder in die Freigabeliste.

4.5 Tests — `scripts/test_version_monitor.py` (neu)

Mit ersetzten Systemaufrufen, ohne Netz, ohne Installation:

1. apt-Parser: englische Ausgabe, `(none)`, fehlendes Paket.
2. apt-Vergleich: `7:7.1.5-0+deb13u1` gegen sich selbst → kein Update; gegen `8:1.0-1` → Update **und** Major.
3. `github_release`: Marke vorhanden → Markenvergleich; Marke leer → Datumsweg.
4. `docker_image`: fehlendes Etikett → „Quelle n/a“, **nicht** „aktuell“.
5. Modelle: unbekannte Kennung → Meldung; keine Quelle → „Quelle n/a“.
6. Fälligkeit: 27 Tage → nicht fällig, 28 Tage → fällig, Quittung setzt zurück.
7. Updater-Filter: eine `apt`-Komponente taucht **nicht** in der Freigabeliste auf.

Eintragen in `scripts/regressiontest.sh` analog Z. 61:

```
run "Versions-Monitor Pruefarten" "$PY" scripts/  
    test_version_monitor.py
```

4.6 Doku-Spiegel

- `ABHAENGIGKEITEN.md` **Z. 75** (Monitor): neue Prüfarten und Quellen, Prüfbefehl ergänzen; **Z. 72** (Updater): den Filter `INSTALLIERBAR` als Invariante festhalten („erkennbar ≠ installierbar“).
- `MIGRATION.md`: Abschnitt 5.21 um **5.21-E** fortschreiben.

5. Sicherheit und Datenschutz

Ampel: grün. Im Einzelnen:

- Alle neuen Quellen sind **reine Leseabrufe ohne Anmeldung**; es verlässt kein Datum des Systems den Server. Der apt-Weg ist vollständig lokal.
- **Bewusst kein Freitext-Befehl im Register.** Der ursprünglich vorgesehene `current_cmd` hätte aus dem Register einen Ausführungsweg gemacht. Statt dessen feste Felder, aus denen der Code den Befehl selbst baut — als Argumentliste, **nie über eine Shell**. Damit bleibt 8.7 gewahrt.

- **GitHub ohne Anmeldung:** 60 Abrufe je Stunde und Adresse. Bei einem Lauf je Woche unkritisch; bei Überschreitung meldet der Monitor „Quelle n/a“ statt einer Falschaussage.
 - **Kein Modelllauf** in irgendeinem der neuen Wege — die Leitplanke aus Abschnitt D der AGB-Faktensammlung bleibt unangetastet.
 - Der Modell-Endpunkt würde einen Schlüssel verwenden, wenn einer vorliegt: **Auflistung verbraucht keine Token**, kostet also nichts. Fehlt der Schlüssel, greift die öffentliche Übersichtsseite.
-

6. Aufwand

Stück	Umfang
version_monitor.py	~130 Zeilen (vier Prüffarten, Fälligkeit, Vergleichsweiche)
updater.py	2 Zeilen
components.json	9 Einträge, 2 Notizen
bot.py	~30 Zeilen (Rückruf + Knopf)
test_version_monitor.py	~90 Zeilen
Doku	3 Stellen

Ein Arbeitsgang mit Regressionstest davor und danach.

7. Reihenfolge und Abnahme

1. Regressionstest **vorher** laufen lassen, Zahl notieren (Grundlinie, A5).
2. Monitor umbauen, Register umstellen, Updater-Filter korrigieren.
3. `python3 scripts/version_monitor.py` von Hand — Protokoll muss für jede der neun Komponenten eine Zeile zeigen, keine mehr mit `manual` außer `verfahren-medien`. **Als root ausführen**, sonst schlägt der Docker-Abruf fehl.
4. Neuen Test schreiben, in den Regressionstest eintragen, beides grün.
5. Knopf für die Verfahrensprüfung einbauen, Bot neu starten, Knopf einmal drücken.
6. Regressionstest **nachher** — gleiche Zahl plus dem neuen Test.

7. Doku-Spiegel nachziehen, dann Adam kurz melden, was sich für ihn ändert (Antwort: nichts an der Bedienung — nur, dass mehr geprüft wird).

Rückweg: Ein einziger git-Widerruf; es wird nichts installiert, nichts migriert, keine Datei außerhalb des Repos angefasst außer der neuen Zustandsdatei unter `logs/`.

8. Ausbaustufen — was jetzt, was später (Adam, 27.07.2026, 10:23 Uhr)

Stufe 1 — dieser Auftrag

Erkennen läuft von selbst, Anwenden auf Freigabe. Alles Prüfbare prüft der Monitor wöchentlich mit; die Verfahrensprüfung meldet sich monatlich mit Knopf.

Stufe 2 — Vollautomatik der Verfahrensprüfung (zurückgestellt, nicht verworfen)

Adam ausdrücklich: „Trotzdem schmeiß das mal nicht ganz raus, diese Vollautomatisierung — wenige Cent im Monat ist natürlich überschaubar. Brauchen wir jetzt am Anfang nicht, hast Recht, müssen wir jetzt nicht aufweichen.“

- **Weg:** die monatliche Recherche über den kostenpflichtigen Zugang statt über das Abo. Damit entfällt die Grauzone vollständig, und der Knopf wird entbehrlich.
- **Kosten:** wenige Cent im Monat (ein Rechercheauftrag), vor der Umstellung einmal beziffern — Geld-Dialog.
- **Wiedervorlage:** wenn die Einführungszeit vorbei ist und Adam die Abläufe kennt. Bis dahin bleibt Stufe 1 — auch, weil der Knopf ihm die Sichtbarkeit gibt, die er ausdrücklich will.

Stufe 3 — selbsttätige Aktualisierung (eigenes Vorhaben, nicht Teil dieses Auftrags)

Adams Zielbild: „Grundsätzlich darf sich das System auch in einem gesunden Sicherheitsrahmen selbst updaten. Das braucht natürlich dann auch Regeln ... dass keine Funktionen eingeschränkt sein

dürfen dadurch oder gar ausfallen, dass keine Sicherheitslücken entstehen oder Funktionslücken. Feste Regeln, auch Bezugsregeln — für alles, das automatisch läuft.“

Was davon schon steht. Die Updater-Härtung A1–A7 ist der Sache nach bereits dieses Regelwerk, nur eben für den Updater: Grundlinie zuerst messen (A5), Bewertung per Delta statt absolut, exakt die freigegebene Version (A3), Lauf-Schloss (A4), vollständiges Einfrieren der Umgebung vor dem Eingriff (A1), laute und ehrliche Rückmeldung bei gescheitertem Rückbau (A2), Selbst-Widerspruch erkennen (A6), Wiederhol-Schutz (A7). Dazu die Bezugs-Integrität aus `ABHAENGIGKEITEN.md` samt Wächter — Adams „Bezugsregeln“ haben also schon ein Zuhause.

Was fehlt, um von „Freigabe“ auf „selbsttätig“ zu gehen:

1. Die Verallgemeinerung der Härtungsregeln von einem Automaten auf **alle**.
2. Eine ausdrückliche Definition, wann ein Schritt als unschädlich gilt — Adams drei Bedingungen (keine Einschränkung, kein Ausfall, keine Lücke) in prüfbare Form gebracht.
3. Der Nachweis je Lauf, nicht die Absicht: Regressionstest **davor und danach**, eingefrorener Rückweg, automatischer Rückbau bei jeder Abweichung.

Kleinster sinnvoller erster Schritt, wenn Adam ihn will: nur die **grüne** Ampelstufe selbsttätig — Patch- und Nebenversionen, nicht gepinnt, mit grüner Grundlinie davor, Regressionstest danach, automatischem Rückbau bei jedem Fehlschlag und lauter Meldung. Gelb (gepinnt) und Rot (Hauptversionssprung) bleiben bei der Freigabe. Das ist die kleinste Stufe, die Adams drei Bedingungen tatsächlich erfüllt, statt sie nur zu behaupten.

Verknüpfung: Dieses Vorhaben ist die konkrete Ausformung des Grundsatzthemas vom 26.07. („selbstlernend, selbstkorrigierend, selbsttestend“) und gehört mit diesem zusammen ausgearbeitet, nicht daneben.

9. Geltung für alles Künftige (Adam, 27.07.2026, 11:11 Uhr)

Adams Einwand: Was wir hier festhalten, darf nicht nur für den Ist-Stand gelten. „Dann kommt irgendwas Neues, das wir bauen, und dann ist das davon ausgenommen — das darf natürlich nicht sein. Wenn wir neue Komponenten zum System hinzufügen, installieren und so weiter, müssen die genauso behandelt werden **oder besser.**“

Der Einwand trifft. Nachgesehen, nicht vermutet — hier der Ist-Stand:

Was neu dazukommt	Wird es automatisch erfasst?
neues Betriebsskript (<code>scripts/*.py</code>)	Ja. Wächter R2 im Selbstcheck liest den Ordner und meldet jedes Skript ohne Eintrag im Bezugsregister.
neues eigenes Modul im Wurzelverzeichnis	Nur teilweise. Der Wächter prüft eine fest verdrahtete Liste von sieben Namen . Ein achttes Modul fällt durch, bis jemand die Liste erweitert.
neue Komponente (Fremdpaket, Systemwerkzeug, Container, Dienst)	Nein. Für das Komponenten-Register gibt es keinen Vollständigkeits-Prüfer — nur die Bitte „muss eingetragen werden“ im Registerkopf und im Bezugsregister.

Der dritte Fall ist genau der, um den es bei den Handprüfungen geht: Eine Komponente, die niemand einträgt, wird nie geprüft — und niemand merkt es. Die Regel steht seit dem 24.07. da und hat keinen Prüfer. **Eine Regel ohne Prüfer ist eine Bitte** (R2, Repo-Regelwerk) — hier ist sie eine.

9.1 Modul-Wächter dynamisch machen

In `bot.py` (`_c_register_vollstaendig`, Z. 4992 ff.) die feste Namensliste durch einen Verzeichnis-Durchlauf ersetzen — genau so, wie es für die Skripte zwei Zeilen tiefer schon gemacht wird:

```
for p in sorted(_REPO_DIR.glob("*.py")):
    if p.name.startswith("test_") or p.name == "bot.py":
        continue
```

```
if p.name not in inhalt:
    fehlt.append(p.name)
```

Damit trägt der Wächter jedes künftige Modul mit, ohne dass jemand daran denken muss.

9.2 Neuer Wächter: Komponenten-Register-Vollständigkeit

Zweite Selbstcheck-Zeile, deterministisch, ohne Netz. Sie vergleicht, was tatsächlich da ist, mit dem, was im Register steht:

1. **Festgehaltene Pakete:** jeder Eintrag in `requirements.txt` muss eine Entsprechung in `components.json` haben. Fehlt eine → Befund.
2. **Projekteigene Dienste:** `systemctl list-units 'claude-*' 'litellm*' 'searxng*' 'ollama*' — jeder gefundene Dienst muss im Register oder im Bezugsregister vorkommen.`
3. **Container:** `docker ps --format '{{.Image}}'` — jedes laufende Abbild braucht einen Eintrag.

Ausgabe wie beim bestehenden Wächter: eine Zeile, die die Lücke beim Namen nennt. Der erste Lauf wird vermutlich sofort etwas finden — so war es beim Wächter R2 auch (`ampel.py`).

Grenze, bewusst gezogen: Es wird **nicht** gegen alle installierten Systempakete abgeglichen; ein Debian hat über tausend. Geprüft wird gegen das, was wir selbst als Abhängigkeit erklärt haben. Alles andere wäre Rauschen und würde den Wächter unglaublich machen.

9.3 „Oder besser“ — der Standard darf steigen, nicht fallen

Adams Zusatz ernst genommen: Wenn für eine neue Komponente ein **genauerer** Prüfweg existiert als der hier gebaute, wird der genauere genommen. Das Register hält die Art der Prüfung je Komponente fest — es zwingt niemanden auf den kleinsten gemeinsamen Nenner.

9.4 Das Prüfsystem selbst prüfen

Adam: „Auch das System kann man natürlich irgendwann nochmal prüfen.“ Die monatliche Verfahrensprüfung (4.3) bekommt dafür eine zweite Frage neben der nach besseren Verfahren:

Gibt es inzwischen einen Weg ins System, den niemand überwacht — eine neue Art von Komponente, einen neuen Automatismus, eine Regel ohne Prüfer?

Damit prüft sich das Prüfsystem in seinem eigenen Takt mit, statt darauf zu warten, dass es jemandem auffällt.

10. Wenn eine Quelle wegbricht (Adam, 27.07.2026, 11:21 Uhr)

Adams Einwand: Auch Bezugsquellen ändern sich — Anbieter vermeiden das zwar in der Regel, aber es kommt vor. Wenn eine Komponente nicht mehr prüfbar ist, muss das **auffallen**, nicht stillschweigend versanden.

Auch dieser Einwand trifft, und zwar auf einen bestehenden Mangel — nicht auf einen, der erst durch den Umbau entstünde. Im Code nachgesehen:

Befund C — eine tote Quelle wird nie gemeldet. In `main()` (Z. 153-155):

```
if not cur or not latest:
    loglines.append(f"? {name}: ... (Quelle n/a)")
    continue
```

Der Fall landet **nur im Protokoll**. Gemeldet wird ausschließlich, wenn es Updates gibt (Z. 176). Bricht die Quelle einer Komponente weg, steht sie also jede Woche stumm als „Quelle n/a“ in einer Datei, die niemand liest — und die Komponente ist faktisch aus der Überwachung gefallen, ohne dass es jemand merkt. Ein leises Verschwinden ist schlimmer als ein lauter Fehler.

Befund D — alle Fehlergründe sehen gleich aus. `_get_json` (Z. 33-39) fängt `Exception` ab und gibt `None` zurück; `_run` gibt bei jedem Fehler `""`. Damit sind ununterscheidbar: Paket umbenannt (404), Ratengrenze erreicht (403), Netz kurz weg (Zeitüberschreitung), Werkzeug gar nicht installiert, Antwort kein JSON. Der wichtigste Fall — **die Quelle existiert nicht mehr** — sieht aus wie ein Schluckauf.

10.1 Fehlergrund festhalten statt verschlucken

```
def _get_json(url: str) -> tuple[dict | None, str]:
    """(Daten, Grund). Grund ist leer bei Erfolg, sonst kurz und
    auswertbar:
    'http 404', 'http 403', 'timeout', 'kein json', 'netz'."""
    try:
        req = urllib.request.Request(url, headers={"User-Agent":
            "momo-version-monitor"})
        with urllib.request.urlopen(req, timeout=HTTP_TIMEOUT) as
            r:
            return json.loads(r.read().decode("utf-8")), ""
    except urllib.error.HTTPError as e:
        return None, f"http {e.code}"
    except TimeoutError:
        return None, "timeout"
    except json.JSONDecodeError:
        return None, "kein json"
    except Exception as e:
        return None, type(e).__name__.lower()
```

Bestehende Aufrufstellen entsprechend anpassen (vier Stück). Der Grund wandert ins Protokoll **und** in den Befund.

10.2 Blindstelle melden — aber erst bei Wiederholung

In der Zustandsdatei aus 4.1 (f) je Komponente den **letzten erfolgreichen Abruf** festhalten. Daraus zwei Stufen:

- **Ein Aussetzer:** nur Protokoll. Ein einzelner Netzhänger ist kein Ereignis; wer ihn meldet, erzieht Adam dazu, Meldungen zu überlesen.
- **Zwei Läufe hintereinander ohne Erfolg** (also rund zwei Wochen) **oder ein eindeutiger 404:** Meldung, auch wenn es sonst keine Updates gibt.

⚠ Nicht mehr prüfbar: `lobe-chat` — seit 2 Läufen ohne Quelle (`http 404`). Zuletzt erfolgreich am 13.07.2026. Bezugsquelle im Register prüfen.

Ein eindeutiger 404 darf sofort melden: Er bedeutet nicht „gerade nicht erreichbar“, sondern „dort ist nichts mehr“.

10.3 Sichtbarkeit im Normalbetrieb

Jede Meldung des Monitors endet künftig mit einer Zeile:

Alle Quellen erreichbar (Stand 27.07.).

oder, wenn nicht, mit der Liste der stummen. Damit ist der Unterschied zwischen „nichts zu melden“ und „ich melde nichts mehr“ für Adam sichtbar — heute sind beide Zustände von außen gleich.

10.4 Quellenwechsel ist eine Registeränderung

Zieht ein Anbieter um, wird **der Registereintrag** angepasst (Art, Name, Ablage) — eine Stelle, nicht verstreut im Code. Das ist der Grund, warum die neuen Prüffarten ihre Ablage aus dem Register beziehen und nicht fest verdrahtet sind.

11. Die Probe aufs Exempel — rückwärts vom Ziel gedacht

Adam am 27.07. um 11:29 Uhr: Er will nicht bei jedem Vorhaben selbst um die Ecken denken müssen. Seine Methode, in seinen Worten: **vom gewünschten Ergebnis rückwärts fragen** — was ist dafür notwendig, was hängt daran, was könnte es beeinflussen, auch bei Dingen, die man noch nicht kennt.

Statt das nur zuzusagen, ist die Methode hier **einmal angewandt** worden. Das Ziel dieser Vorlage lautet: Adam muss nichts von Hand prüfen und erfährt zuverlässig, was ansteht. Rückwärts gefragt, was dafür alles wahr sein muss:

1. Die Prüfungen laufen → Abschnitt 4.
2. Neue Komponenten werden erfasst → Abschnitt 9.
3. Quellen bleiben erreichbar, sonst fällt es auf → Abschnitt 10.
4. **Der Monitor läuft überhaupt** → offen.
5. **Die Meldung erreicht Adam** → offen.

Punkt 4 und 5 hatte kein Abschnitt abgedeckt. Beide sind im Code nachgesehen:

Befund E — niemand bewacht die Zeitgeber. Der tägliche Funktionscheck prüft die Dienste (`claude-telegram-bot` , `searxng` , `ollama` , `litellm`), aber **keinen einzigen Zeitgeber**. Fällt `claude-version-monitor.timer` aus — durch einen Fehler, ein Systemupdate, ein versehentliches Abschalten —, läuft der Monitor nie wieder, und **von außen ist das nicht von „diese Woche gab es nichts“ zu unterscheiden**. Dieselbe Blindheit gilt für die anderen eigenen Zeitgeber (Hygiene, Tagescheck, Protokoll-Abgleich).

Abhilfe, in den Tagescheck: für jeden projekteigenen Zeitgeber zwei Fragen — ist er aktiv, und wann lief er zuletzt (`systemctl show -p LastTriggerUsec`)? Liegt der letzte Lauf länger zurück als sein Takt plus Toleranz → Befund. Billig, deterministisch, deckt alle künftigen Zeitgeber mit ab (Abschnitt 9 lässt grüßen).

Befund F — ein gescheiterter Versand ist still. In `_send_telegram` (Z. 111–131) wird jeder Sendeversuch mit `except Exception: pass` abgeschlossen; ist die Env-Datei nicht lesbar, kehrt die Funktion wortlos zurück. Findet der Monitor also etwas und der Versand misslingt, ist der Befund verloren — er steht im Protokoll, das niemand liest.

Abhilfe: Der Versand meldet zurück, ob er geglückt ist. Bei Fehlschlag legt der Monitor einen **Merker** ab; der Tagescheck greift ihn auf und meldet ihn (er hat einen eigenen Sendeweg), und der nächste Monitorlauf versucht die Zustellung erneut. Damit ist der einzige Fall abgedeckt, in dem man einen Fehler nicht auf demselben Weg melden kann, auf dem er entstanden ist.

Was daraus folgt — über diese Vorlage hinaus: Die fünf stillen Stellen dieses Auftrags (Befunde B bis F) haben alle dieselbe Signatur. Es ist nie ein lauter Fehler, sondern immer **ein Ausbleiben, das wie Ruhe aussieht**. Danach muss zuerst gesucht werden, nicht nach Abstürzen. Als eigenes Vorhaben angelegt.